3300.01 Final Project: DEAL OR NO DEAL

Names: Anjana Korisal, Alan Heng, Kevin Davis, and Yuna Kitaguchi

Introduction:

For our final project we designed and implemented a Deal or No Deal system on the Nexys A7 FPGA. Our system allows the user to perform multiple function within the FPFS including selecting numbered cases, tracking game progression, and interact with a simulated banker who provides offers based on the current game state. The system handles everything from input selection to game logic to output display. It uses switches and buttons for user input, and displays information using LEDs, a 7-segment display, and UART communication to a PC (C# UI). The design also includes FIFO input buffering, FSM control, and modular Verilog structure.

System Overview:

This project implements a simplified version of Deal or No Deal where the player selects from 16 cases. Each case has a fixed money value.

The player first selects one case to keep. After that, they continue opening other cases. As the game progresses, the system calculates the remaining prize values and generates banker offers at certain points.

The system is built using multiple modules:

- FIFO (handles input buffering)
- Case selector (checks if a case is valid)
- FSM (controls game flow)
- Prize logic (tracks money values)
- Bank logic (calculates offers)
- Display and UART (outputs game state)

Everything is connected through a top module that manages how the system runs.

I/O:

Inputs

- Switches (SW[3:0]) - Used to choose a case number (0–15)
- Push Button (case_confirm) - Confirms the selected case
- Push Buttons (Deal / No Deal)- Used to respond to banker offers
    - Deal - accept offer and end game
    - No Deal - continue opening cases
- Reset Button (rst) - Resets the game

Outputs

- LEDs - Show which cases are still available
    - ON = available, OFF = already opened
- 7-Segment Display
    - Current selected case
    - Preview case (before confirming)
- UART
    - Sends messages to a PC interface (C# UI)
- VGA
    - .....

FIFO:

The FIFO is used to store case selections before they are processed by the rest of the system. For is it is used to separate button timing from logic timing, prevent issues from button bouncing or fast inputs, ensure inputs are processed in order. In our game we use it to start the confirmed case inputs from the player and send data to the case selector when it's ready.

Case Selector:

The case selector is responsible for validating case selections and keeping track of which cases are still available in the game. It reads case values from the FIFO and processes them one at a time.

When a case is received, the module checks it against the available_cases register. If the case has not been opened yet, it is marked as unavailable and a valid_selection signal is generated. If the case has already been selected before, the system instead generates an invalid_selection signal.

At the same time, the module continuously updates the available_cases output, which is connected to the LEDs to visually show which cases are still remaining. This allows the player to see the game state directly on the board.

The case selector ensures that each case can only be chosen once and that all selections are properly validated before being used by the FSM.

FSM(Finite State Machine):

The FSM controls the overall game flow and determines what stage the game is in at any given time. It acts as the main controller that decides how the system should respond to user inputs and internal signals.

At the start of the game, the first valid case selected becomes the player's personal case. After that, any additional valid selections are treated as open cases. As more cases are opened, the FSM keeps track of progress and triggers a banker's offer after a certain number of rounds.

When the system reaches the banker stage, it waits for input from the Deal or No Deal buttons. If the player presses Deal, the FSM ends the game, and the player accepts the banker's offer. If the player presses No Deal, the game continues, and more cases can be opened, until there are no more cases left to open besides their own.

These decisions are controlled using deal_pulse and no_deal_pulse signals, which are short pulses to ensure that each button press only causes a single state transition. The FSM ensures that the game progresses in the correct order and that all transitions happen reliably.

Prize Logic:

The prize logic assigns a fixed money value to each case and keeps track of the remaining prize pool throughout the game.

Each case number is mapped to a predefined value using a lookup table. When a valid case is selected, its value is revealed and removed from the remaining prize pool. The system then recalculates the total and average of the remaining values.

This updated information is important because it directly affects the banker's offer. By continuously updating the remaining total and average, the prize logic keeps the game accurate and consistent.

Bank Logic:

The bank logic is responsible for calculating the banker's offer based on the current state of the game. It takes in the remaining_average, opened_case_count, and a generate_offer control signal, and outputs the banker_offer along with an offer_valid signal.

The offer is calculated using the formula:

Banker Offer = Remaining Average × Risk Percentage

The risk percentage is determined based on the number of opened cases, allowing the offer to change as the game progresses. When generate_offer is asserted, the module computes the offer using arithmetic operations and updates the result on the clock edge. The output is stored in registers to keep it synchronized with the rest of the system.

This module demonstrates proper Verilog design by combining conditional logic, arithmetic operations, and synchronous updates while keeping the calculation isolated from the main control logic

Display and Communication:

The project uses several display and communication interfaces so that the game state can be shown clearly both on the FPGA board and through external displays. The VGA output provides the main hardware-based visual interface by displaying the Deal or No Deal game board on a monitor and showing the status of cases as they are opened or remain available. UART communication is used to send game updates from the FPGA to a C#

graphical interface on the computer. These UART messages include information such as the previewed case, confirmed case, invalid selections, prize values, and banker offer amounts. In addition to VGA and UART, the LEDs and 7-segment display provide board-level feedback. The LEDs represent which cases are still available, while the 7-segment display shows the preview case and the currently selected case. Together, these interfaces make the system easier to demonstrate and debug because the same game behavior can be observed through the monitor, the computer UI, and the physical FPGA board.

Testbench/Simulation Strategy:

A testbench was created to verify the full system before running it on hardware. The main testbench simulates the top-level design, including switches, case confirm button, Deal/No Deal buttons, and outputs such as LEDs, 7-segment display, UART, and VGA signals.

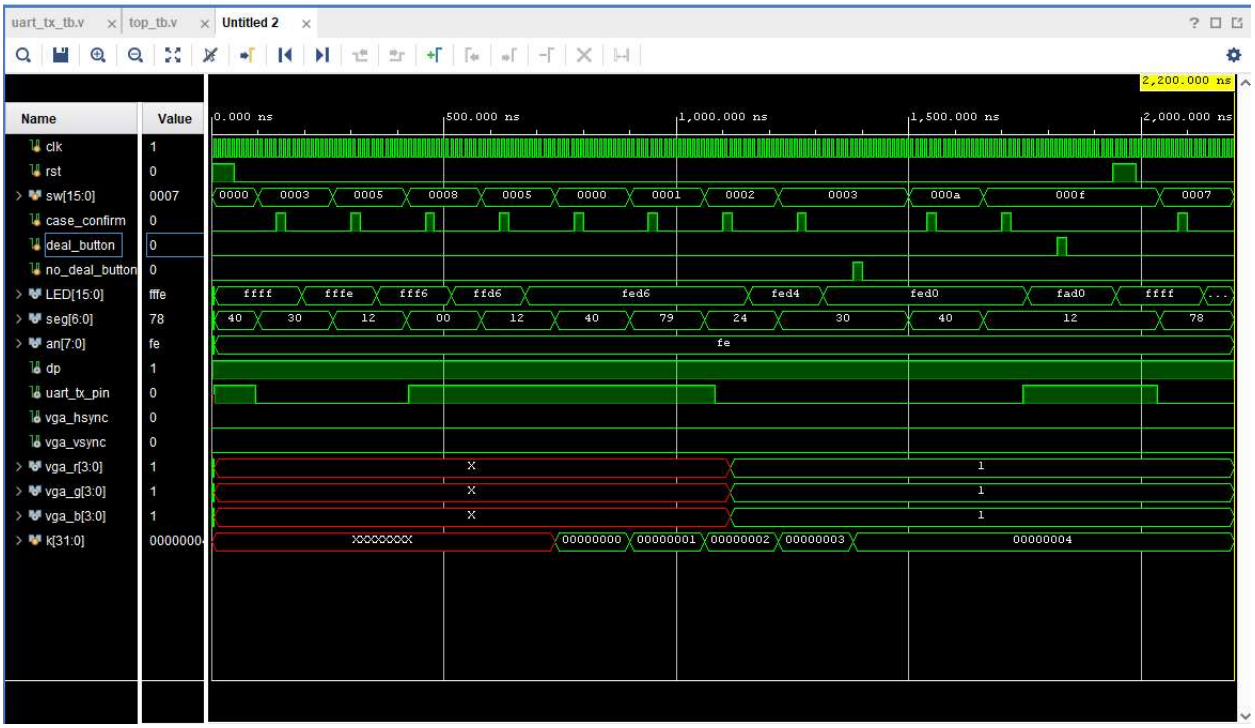
The test sequence includes selecting a player case, opening multiple cases, attempting a duplicate selection to check invalid behavior, and testing both Deal and No Deal inputs to verify FSM transitions. A for loop is used to automate repeated case selections, and $monitor is used to track important internal signals such as FSM state, selected case, opened case count, banker offer, and game status.

A separate UART testbench was also used to verify correct transmission of characters and proper handling of the send, tx, and busy signals.

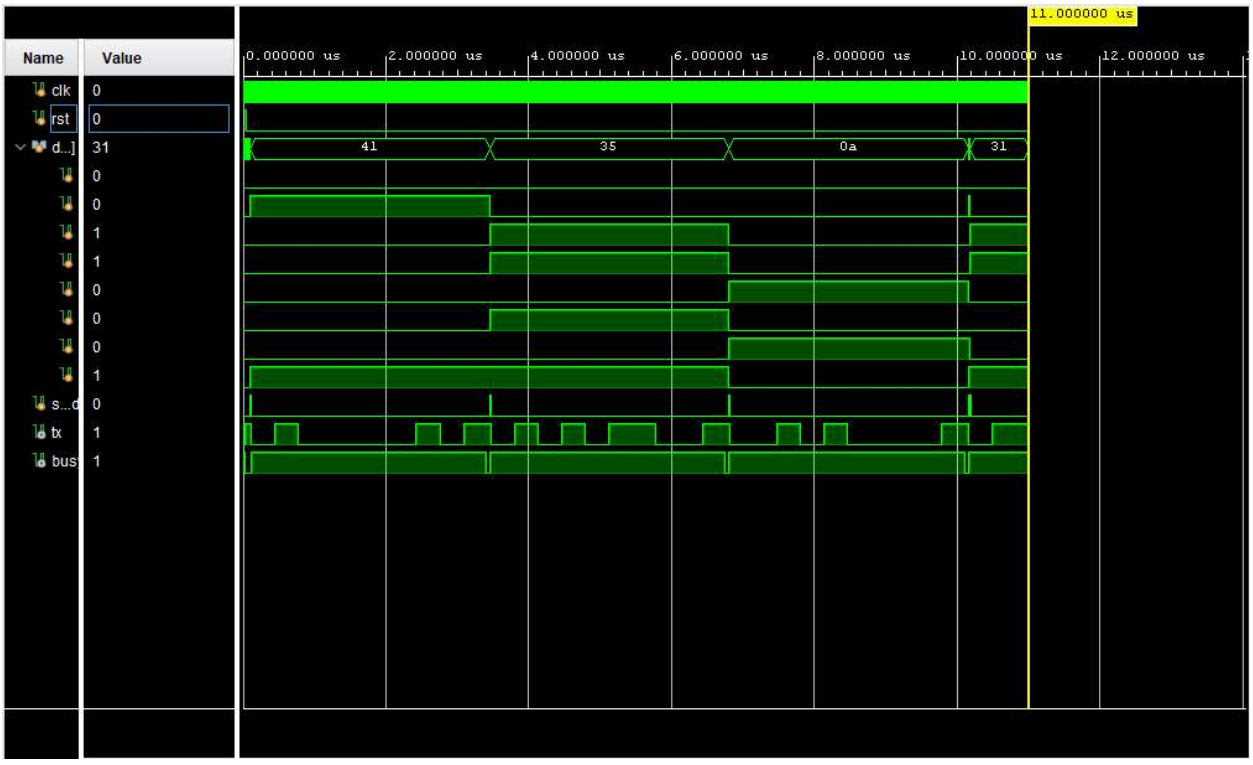
Overall, the testbenches confirm correct system behavior, including input handling, game progression, duplicate detection, and UART communication before hardware implementation.

Simulation Screenshot:

Master Top_tb Simulation



UART_tx_tb simulation



Modeling Style:

Our project uses a modular Verilog design because the Deal or No Deal system has several different hardware functions, including input handling, game logic, prize tracking, display output, UART, and VGA. Splitting the design into separate modules made the project easier to debug, test, and explain.

Structural Modeling

We used structural modeling in the top-level module. The top module connects the FIFO, case selector, prize logic, banker logic, UART transmitter, VGA module, LEDs, and 7-segment display. This keeps the design organized because each module has one clear responsibility.

Behavioral Modeling

We used behavioral modeling for logic that changes over time, such as the case selector, FSM, prize tracking, and UART message control. These parts need registers because the system must remember which cases have already been opened, what the current selected case is, and what game state the player is in.

Dataflow Modeling

We used dataflow modeling for simpler signal relationships, such as assigning switch values into the FIFO input, connecting available cases to LEDs, and creating control signals. These are direct relationships, so continuous assign statements made the code cleaner.

Conclusion:

This project successfully implements a working version of *Deal or No Deal* on an FPGA.

It demonstrates the use of FIFO input handling, FSM-based control, and player interaction through both case selection and Deal or No Deal decisions. The design is modular and uses multiple FPGA I/O components effectively.

Overall, the system works in both simulation and hardware, showing a complete digital design from input to output and demonstrating a strong understanding of FPGA-based system design.